基于 Qwen2.5-7B 的医疗领域 LoRA 微调实战报告

AI 面试/课程实战·A4 项目部署微调全流程

一、项目概述与微调原因

（一）项目背景

本项目为“AI 面试/课程实战·A4 项目部署微调”实操任务，旨在独立完成一条从数据准备、环境部署、模型微调到线上服务与展示的完整链路。项目以医疗问答为垂直场景，选用 Qwen2.5-7B-Instruct 作为基座模型，以 Huatuo-26M 医疗问答数据集为训练语料，通过 LoRA 参数高效微调获得面向医疗领域的专用对话模型，并先后完成模型合并、vLLM 推理服务部署、Flask 接口封装以及展示平台在线测试，最终形成可演示、可提交的完整成果。

（二）微调原因

通用大语言模型虽具备较强的语言理解与生成能力，但在垂直领域问答中仍存在明显不足：在医疗场景下，模型可能缺乏领域术语的准确理解，回答不够规范，甚至给出缺乏安全边界的建议。为使模型具备医疗问答所需的专业知识与表达规范，需要以高质量医疗语料进行领域微调。考虑到全参微调对算力与显存要求高、训练周期长，本项目采用 LoRA 低秩适配方法，仅训练约 2018 万个参数（约占模型全部参数 76.16 亿的 0.265%），显著降低训练成本，同时保持接近全参微调的领域适配效果。基座模型选择 Qwen2.5-7B-Instruct，主要基于三点考虑：其一，该模型中文能力与指令遵循能力较强，适合中文医疗问答；其二，课程文档与配套工具（LlamaFactory、vLLM、展示平台）对该模型提供了完整的链路支持；其三，7B 参数量级在单卡 A800 上兼顾效果与部署可行性。

二、数据集来源

（一）Huatuo-26M 数据集

本项目训练数据来源于 Huatuo-26M 开源医疗问答数据集。该数据集以中国古代名医“华佗”命名，包含约 2600 万条医学问答数据，覆盖疾病症状、诊疗建议、用药安全、饮食护理等常见医疗咨询主题，是中文医疗领域公开数据集中规模较大、质量较受认可的资源之一。

（二）本项目实际使用的数据

考虑到训练时间与算力约束，本项目从数据集中整理出训练集 10000 条、验证集 1000 条、测试集 1000 条，统一转换为 alpaca 三字段格式，即每条样本由 instruction（指令）、input（输入）、output（回答）三个字段构成。训练集规模超出课程文档建议的 7000 至 8000 条，因此在训练配置中通过 max_samples 参数将实际参与训练的样本数控制在 8000 条，验证集用于训练过程中的损失评估。

三、数据整理（清洗）流程

（一）格式转换

原始 Huatuo 数据以问答对形式存在，为使数据适配 LlamaFactory 的标准训练流程，首先将其转换为 alpaca 格式，将问题文本填入 instruction 与 input 字段，将标准医学回答写入 output 字段，保证字段语义明确、样本结构统一，同时确保训练阶段指令模板能够正确拼接。

（二）质量核查

在数据清洗阶段，通过脚本对全部样本进行了字段完整性与空值检查，确认训练、验证、测试三份数据均无缺失字段、无空值样本；同时将训练集与测试集进行去重比对，避免同一问题样本同时出现在训练与测试阶段而影响评估可信度。

（三）数据划分与注册

数据按 8：1：1 的比例划分为训练、验证、测试三份，上传至服务器后注册进 LlamaFactory 的 dataset_info.json 配置文件中，分别以 huatuo_train、huatuo_valid、huatuo_test 作为数据集名称，训练与评估阶段直接按名称引用，避免了重复处理与配置错误。

四、微调方法

（一）实验环境

实验在广州超算平台的 A800 80GB 单卡实例上进行，微调框架采用 LlamaFactory 0.9.6.dev0，Python 环境为 3.11，主要依赖 torch 2.14.0、transformers 5.8.0、peft 0.18.1、trl 0.24.0，环境依赖通过清华镜像源安装。具体环境配置如表 1 所示。

表 1 实验环境配置

项目	配置
服务器	广州超算 runner 实例
GPU	NVIDIA A800 80GB PCIe（单卡）
微调框架	LlamaFactory 0.9.6.dev0
Python	3.11（conda 环境 llamafactory）
关键依赖	torch 2.14.0 / transformers 5.8.0 / peft 0.18.1 / trl 0.24.0

（二）LoRA 原理简述

LoRA（Low-Rank Adaptation）通过在原始权重旁引入低秩增量矩阵对模型进行适配，训练过程中冻结基座模型权重，仅更新低秩矩阵参数，从而以极小的参数量完成领域知识注入。本项目 lora_target 设置为 all，即对模型全部线性层施加低秩适配，以尽可能充分地利用可训练容量。

（三）训练配置

训练基于 LlamaFactory 的 LoRA 指令微调配置（qwen3_lora_sft.yaml）执行，主要超参数如表 2 所示。

表 2 LoRA 微调主要配置

配置项	取值
基座模型	Qwen2.5-7B-Instruct
训练数据	huatuo_train（max_samples=8000）
验证数据	huatuo_valid
lora_rank	8
lora_alpha	32
lora_target	all
learning_rate	2.0e-5
num_train_epochs	3
精度	bf16
输出目录	~/output/qwen2.5-7b-lora

（四）训练与评估

模型在 8000 条样本上完成 3 个 epoch 共 3000 个优化步的训练，可训练参数为 20185088 个。训练过程中记录并保存了训练损失与验证损失曲线；最终在验证集上的评估损失（eval_loss）为 1.6915，较训练初期明显下降，表明模型对医疗问答任务产生了有效适配。

（五）模型合并

训练完成后，使用 LlamaFactory 的模型导出功能将 LoRA 适配器与基座权重合并，在 CPU 上导出为完整的微调模型（qwen2.5-7b-merged），得到 config.json、4 个 safetensors 权重分片及 tokenizer 文件，作为后续 vLLM 部署的模型来源。

五、模型部署与平台展示

（一）vLLM 部署

合并后的模型通过 vLLM 0.8.5 部署为 OpenAI 兼容的推理服务，监听 22222 端口，served-model-name 设为 qwen2.5-7b-medical，采用 half 精度并设置 gpu-memory-utilization 为 0.3。部署过程中解决了版本兼容与 tokenizer 配置问题，服务启动后/v1/models 与/v1/chat/completions 接口均返回正常，实测医疗问答输出专业规范。

（二）Flask 服务端

为满足展示平台对/chat 接口的调用要求，基于 Flask 开发了服务端程序，内部通过 OpenAI 客户端连接本机 vLLM 服务（127.0.0.1:22222/v1），对外监听 20003 端口，接收平台下发的 query 字段并返回模型回答；服务端开启 threaded=True 多线程模式，支持并发处理多个请求。

（三）展示平台配置

在 OJAC 微调展示平台（https://aiqnmsg.cn）创建智能体，将经助教申请的公网转发地址填写为微调后 API（https://262c5a7cd8a04506b7d6dae5f5b22b0f.proxy.nscc-gz.cn:8443/chat），智能体设为非公开，并在平台完成对话测试，平台可正常返回模型的医疗回答，全链路验证通过。

六、微调前后对比

（一）训练损失对比

从训练过程看，模型在医疗问答验证集上的损失由训练初期的高位逐步下降，最终收敛至 1.6915，说明微调使模型在医疗数据上的预测能力得到实质提升；训练过程保存的训练损失与验证损失曲线完整记录了损失下降趋势。

（二）微调后回答示例

以“高血压患者饮食要注意什么？”为例，微调后模型给出了规范、安全的回答：“高血压患者的饮食应该注意以下几点：低盐饮食，减少食盐摄入量，成人每日食盐摄入量不超过 5 克，避免食用腌制、熏烤等高盐食品；控制体重，保持健康体重，超重或肥胖会增加心脏负担，不利于血压控制；均衡膳食，多吃新鲜蔬菜和水果……请注意，以上建议仅供参考，在实施任何饮食计划之前，请务必咨询医生或其他医疗专业人员的意见。”回答结构完整、要点准确，并包含就医提醒，符合医疗问答场景的安全要求。

（三）微调前回答对比说明

微调前模型为通用底座，未注入医疗领域知识。为直观对比，可在服务器上加载未微调的 Qwen2.5-7B-Instruct 模型，对同一问题发起推理，与上述微调后回答进行对照；由于本项目按课程要求仅配置微调后 API，平台微调前对比区未启用，本报告以训练损失变化与微调后回答质量作为微调效果的证据。

七、总结与后续计划

（一）成果总结

本项目完整跑通了“数据准备—环境部署—LoRA 微调—模型合并—vLLM 部署—Flask 封装—平台展示”全流程，最终得到一个可在公网对话的医疗问答智能体。微调后模型在医疗问答中表现专业、规范，回答包含用药与就医安全提示，具备实际演示价值。

（二）问题与改进

项目实施中主要遇到两类问题：一是 vLLM 与依赖版本兼容问题，通过锁定 vllm 0.8.5 与配套依赖版本解决；二是合并模型 tokenizer 配置格式问题，通过调整 tokenizer_config.json 的 extra_special_tokens 字段格式解决。后续可从扩充训练数据量、调整 LoRA 秩与学习率、增加领域评测集等方向进一步优化模型效果。